

Chapter 1 English Study Notes

1.2 Eight Great Ideas in Computer Architecture + 1.3 Below Your Program

Purpose. These notes explain the main ideas from Chapter 1 in clear academic English. The focus is on how high-level programs become executable machine code and how this connects to computer architecture.

1.2 Eight Great Ideas in Computer Architecture

1. Design for Moore's Law

Computer architecture must be designed for the future. Since hardware technology improves over time, architects need to anticipate that future chips may contain more transistors, more memory, and more processing power.

Key idea: Good computer design should not only fit today's technology, but also prepare for tomorrow's technology.

2. Use Abstraction to Simplify Design

Abstraction means hiding unnecessary details so that people can think at a higher level. For example, when programmers write code in C, Java, or Python, they do not need to directly control transistors, circuits, or binary signals.

Key idea: Abstraction helps us manage complexity.

3. Make the Common Case Fast

Computer systems should optimize the operations that happen most frequently. If a CPU often performs arithmetic operations or memory access, engineers should make those common operations very fast.

Key idea: Optimizing the common case usually improves overall performance the most.

4. Performance via Parallelism

Parallelism means doing multiple tasks at the same time. Instead of having one worker complete all the work, the system can use multiple workers to increase performance.

Key idea: More workers can improve performance when the work can be divided.

5. Performance via Pipelining

Pipelining divides a task into several stages. Different stages can work at the same time on different parts of the task. A good example is an assembly line in a factory.

Key idea: Pipelining improves throughput by overlapping the execution of multiple tasks.

6. Performance via Prediction

Prediction allows a computer to guess what will happen next and start working early. If the prediction is correct, performance improves. If the prediction is wrong, the system must recover.

Key idea: Prediction can improve performance when mistakes are recoverable.

7. Hierarchy of Memories

Computer memory is organized into levels. Small and fast memory is used for frequently accessed data. Large and slow memory is used for less frequently accessed data.

Registers -> Cache -> Main Memory -> SSD/HDD

Key idea: Memory hierarchy gives the illusion of large, fast, and affordable memory.

8. Dependability via Redundancy

Important systems should not assume that hardware will never fail. Instead, systems should include backup components so that they can continue working even if one part fails.

RAID
Backup servers
ECC memory
Cloud replication

Key idea: Redundancy improves reliability.

1.3 Below Your Program

Main Idea

Computers do not directly understand C, Java, or Python code. At the hardware level, computers only understand patterns of 0s and 1s.

A high-level program must be translated and managed by several layers before the CPU can execute it.

High-level language
-> Compilation system
-> Assembly language
-> Machine language
-> Operating system
-> Memory
-> CPU / Processor
-> I/O devices

Examples of I/O devices include the screen, keyboard, disk, and network card.

Example C Program

We can use this C program to understand what happens below the program:

```
#include <stdio.h>

int add(int a, int b);

int main() {
    int result = add(3, 4);
    printf("%d\n", result);
    return 0;
}
```

This program uses:

main() -> written by the programmer
add() -> defined in another source file
printf() -> provided by the C standard library

The Compilation System

A C program usually goes through four major stages before it becomes an executable program.

main.c
-> Preprocessing
main.i
-> Compilation
main.s
-> Assembly

```
main.o
-> Linking
executable file
```

Stage	Input	Output	Main Job
1. Preprocessing	main.c	main.i	Preprocessing prepares an expanded version of the source code.
2. Compilation	main.i	main.s	Compilation translates high-level language into assembly language.
3. Assembly	main.s	main.o	Assembly turns symbolic instructions into binary machine code.
4. Linking	main.o + math.o + libraries	executable file	Linking connects separately compiled pieces of machine code into one executable program.

1. Preprocessing

The preprocessor handles lines such as `#include <stdio.h>`. It expands header files and macros.

```
Input:  main.c
Output: main.i
```

Key idea: Preprocessing prepares an expanded version of the source code.

2. Compilation

The compiler translates the preprocessed C code into assembly language. Assembly language is a low-level symbolic language that is closer to machine instructions than C code.

```
Input:  main.i
Output: main.s
```

Key idea: Compilation translates high-level language into assembly language.

3. Assembly

The assembler translates assembly language into machine code. The `.o` file is called a relocatable object file. It already contains machine code, but it is not a complete executable program yet.

```
Input:  main.s
Output: main.o
```

Key idea: Assembly turns symbolic instructions into binary machine code.

4. Linking

The linker combines object files and libraries to create a complete executable program.

```
Input:  main.o + math.o + libraries
Output: executable file
```

Key idea: Linking connects separately compiled pieces of machine code into one executable program.

Why Linking Is Needed

Suppose the program is divided into two files.

main.c

```
#include <stdio.h>
```

```
int add(int a, int b);

int main() {
    int result = add(3, 4);
    printf("%d\n", result);
    return 0;
}
```

math.c

```
int add(int a, int b) {
    return a + b;
}
```

After compilation:

```
main.c -> main.o
math.c -> math.o
```

At this point:

```
main.o defines main
main.o needs add
main.o needs printf
```

```
math.o defines add
libc defines printf
```

The linker connects them:

```
main.o needs add    -> math.o provides add
main.o needs printf -> libc provides printf
```

What the Linker Does

The linker mainly does two important jobs:

1. Symbol resolution
2. Relocation

1. Symbol Resolution

A symbol is usually a function name or a variable name. Examples include main, add, printf, and result.

Symbol resolution means finding out where each symbol is defined.

```
main.o needs add
math.o provides add

main.o needs printf
libc provides printf
```

Key idea: Symbol resolution answers the question: Who provides this function or variable?

2. Relocation

Relocation means fixing addresses after the linker knows where every function and variable will be placed.

In main.o, the call to add() may originally look like this:

```
call add
```

But the CPU does not understand the name add. The CPU needs an address or an offset. Before linking, the address is unknown:

```
call ????
```

After linking, the linker knows where add() is located. For example:

```
main is placed at 0x401000
add is placed at 0x401050
```

```
printf entry is placed at 0x401080
the string "%d\n" is placed at 0x402000
```

Relocation Example

On many x86-64 systems, a call instruction often uses a relative offset.

```
Address of call instruction: 0x401020
Address of next instruction: 0x401025
Address of add function:      0x401050
```

The CPU uses this rule:

```
Target address = address of next instruction + offset
```

So the linker calculates:

```
offset = address of add - address of next instruction
        = 0x401050 - 0x401025
        = 0x2B
```

Then the linker patches the machine code as:

```
call +0x2B
```

When the CPU executes it:

```
0x401025 + 0x2B = 0x401050
```

Key idea: Relocation patches unknown addresses or offsets in the machine code.

Linking vs. Loading

Concept	What It Does	Simple Path
Linking	The linker creates the executable file.	main.o + math.o + libc -> executable
Loading	The operating system loads the executable file into memory.	executable -> memory -> CPU starts executing

Important: After linking, the program has not started running yet. The program starts running only after the operating system loads it into memory.

Final Summary

The full process is:

```
Source code
-> Preprocessing
-> Compilation
-> Assembly
-> Linking
-> Executable file
-> OS loads program into memory
-> CPU executes machine instructions
-> I/O devices show the result
```

A shorter version:

```
main.c -> main.i -> main.s -> main.o -> executable -> memory -> CPU -> output
```

Summary

The most important idea in this chapter is: A program written by a human is not directly executed by the hardware. It must pass through a compilation system, be linked with other object files and libraries, loaded into memory by the operating system, and finally executed by the CPU as machine instructions.

In simple words:

Compiler translates.
Assembler encodes.
Linker connects.
Loader loads.
CPU executes.

Even shorter:

Translation -> Connection -> Loading -> Execution

This is the real meaning of Below Your Program.